

A build fails at 2am — and someone has to **read the logs**

- ✗ CI logs are **huge and noisy** — the real error is buried in thousands of lines.
- ✗ Triage is **manual and repetitive**: open the run → find the failed job → scroll the log → guess the cause.
- ✗ The same failures recur — **test breaks, bad dependency, flaky, OOM** — but each is re-diagnosed from scratch.
- ✗ Whoever's on call is **blocked**, and the fix + a clear write-up rarely land on the PR.

The gap: the *reading and root-causing* is slow and mechanical — but a human should still decide and sign off. Automate the first without losing the second.

A **LangGraph agent** that triages the failure for you

Point it at your repo. It lists the **real failed GitHub Actions runs**, investigates one live — reading the actual job logs — and gpt-4o returns a **structured triage**: root cause, the exact evidence line, and a concrete fix. Then it posts that back on the commit, once a human approves.

01

Investigate

ReAct loop: the agent reads jobs + real logs itself.

02

Diagnose

Category · root cause · evidence line
· fix · confidence.

03

Explain

Structured triage a human can trust
— grounded in the logs.

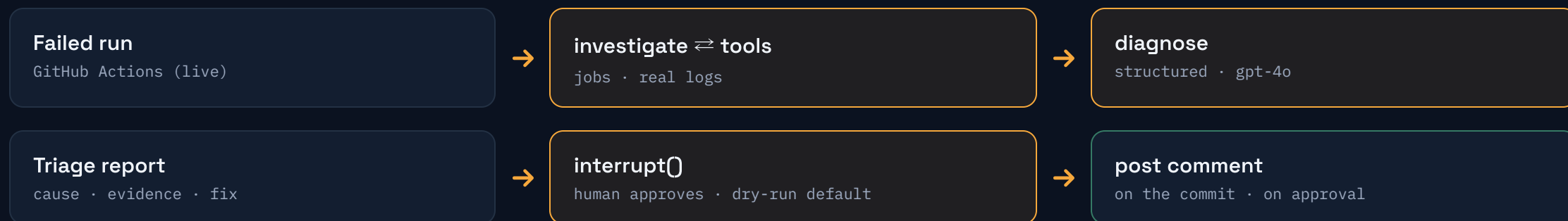
04

Post

Human-approved comment on the commit. Dry-run by default.

Production-standard — no mock data. It reads the real GitHub API and the real logs your pipeline produced. The LLM only reads and reasons; the one write is a comment, behind an explicit approval.

The **process**, end to end



Built with LangGraph: `StateGraph` + `ToolNode` + `interrupt()` for human-in-the-loop. Connects to OpenAI (gpt-4o) and the GitHub REST API (token or `gh`). Launch: `streamlit run app.py`.